

Lab 9

This lab covers material on

- Graphs
- Neural Networks
- Optimization

Learning Objectives

- Understand the mathematical underpinnings of backpropagation
- Be able to create a neural network step-by-step with python code
- understand loss over training epochs and train/test/validation splits

Submission And Assignment Instructions

Submit your `Lab9_lastname_firstname.pdf` file on canvas with your answers clearly marked (✓) and code commented. Skeleton code and/or comments are provided throughout to help you.

- ? Questions you must answer or tasks are marked with a " ? "
- ✓ Answers you should give are marked with "✓ Answer:"
- *Helpful hints are usually given in italics*
- Code you need to write is marked with `## ? YOUR CODE HERE`
 - ★ Bonus questions are marked with a "★ "
 - 🍓 Bonus answers are marked with a "🍓 "

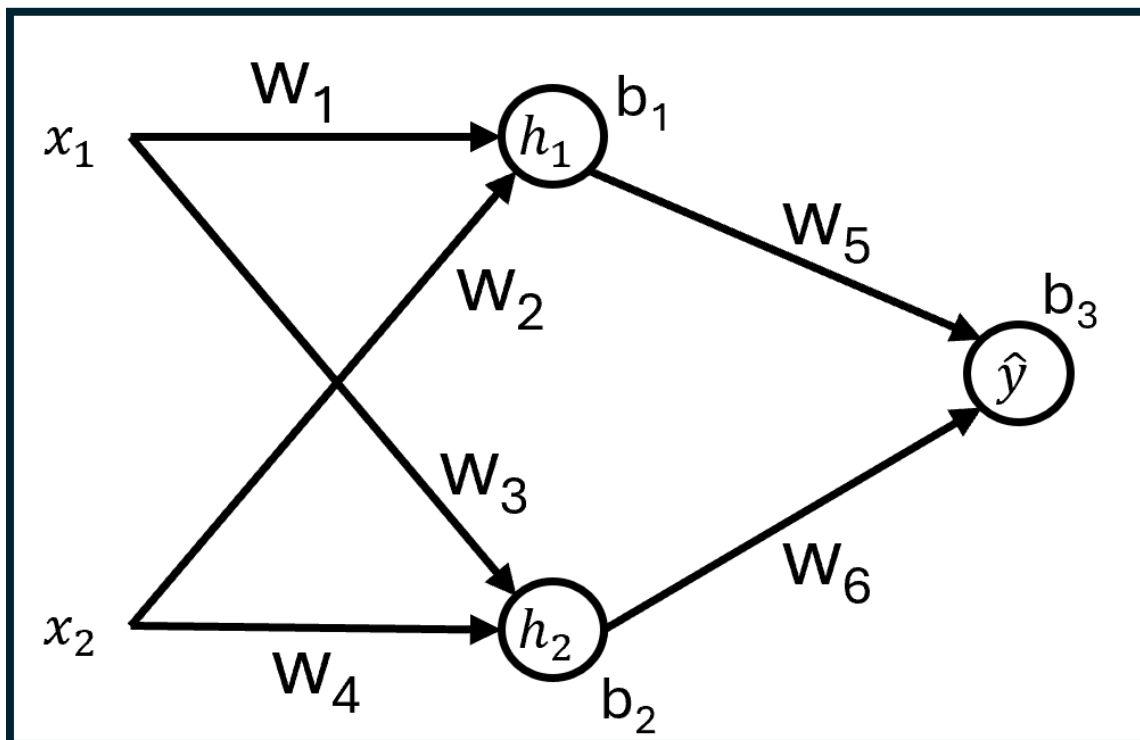
Please don't erase ANY of these markings! They're to help the TAs find your answers and code as much as they're to help you find the questions.

Section 1: Backpropagation Math

We're going to walk through a couple steps of backpropagation using a very simple feed-forward network (pictured below), to help us understand why backpropagation is useful.

The network below has one hidden layer and a single output. **Each neuron also has an activation function, f , that is not pictured. For example, the output value of the top neuron in the hidden layer is $h_1 = f(w_1x_1 + w_2x_2 + b_1)$. This activation function is also applied to the output layer, e.g. $\hat{y} = f(w_5h_1 + w_6h_2 + b_3)$.**

This is what the network looks like:



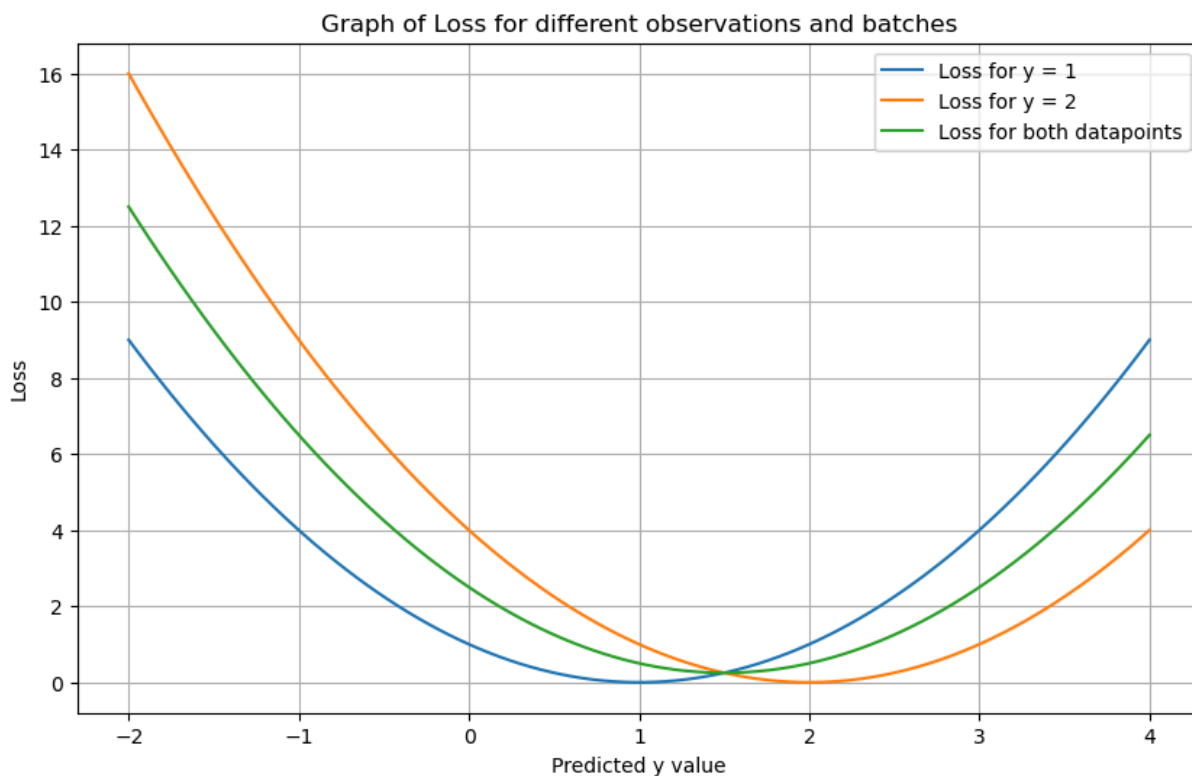
? 1.1 Let's call the loss function L and choose it to be the mean squared error function (MSE). Write the loss function for n datapoints in terms of y and $\hat{y}$, the predicted output value for each input vector $\vec{x}$.

```
In [94]: from IPython.display import Image, display
display(Image("1.1.png"))
```

$$1.1) \quad L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

? 1.2 Graph or sketch an estimate of loss (L) as a function of $\hat{y}$ for one training datapoint that has $y = 1$. Add a line or make a second graph for a second training observation that has a true value of $y = 2$. Add a line or make a third plot to graph loss as a function of both datapoints. Label your sketches or plots clearly. Is your estimate for the loss function different for different training data points? For a given value of $\hat{y} = 1$, is the estimate of $\frac{\partial L}{\partial \hat{y}}$ different for different observations? Does the choice of observation affect my estimation of the loss function and its gradient during stochastic gradient descent?

```
In [95]: ## YOU DON'T HAVE TO USE THIS CODE, YOU CAN SKETCH IT!  
# You can also take a screenshot of some other graphing software  
  
import numpy as np  
import matplotlib.pyplot as plt  
  
# Define the functions  
def f(x):  
    # LOSS FOR Y=1  
    f = (1-x)**2  
    return f  
  
def g(x):  
    # LOSS FOR Y=2  
    g = (2-x)**2  
    return g  
  
def h(x):  
    # LOSS FOR BOTH  
    h = ((1 - x)**2 + (2 - x)**2) / 2  
    return h  
  
# Create x values  
x = np.linspace(-2, 4, 100)  
  
# Calculate y values for each function  
y_f = f(x)  
y_g = g(x)  
y_h = h(x)  
  
# Create the plot  
plt.figure(figsize=(10, 6))  
plt.plot(x, y_f, label='Loss for y = 1')  
plt.plot(x, y_g, label='Loss for y = 2')  
plt.plot(x, y_h, label='Loss for both datapoints')  
plt.xlabel('Predicted y value')  
plt.ylabel('Loss')  
plt.title('Graph of Loss for different observations and batches')  
plt.legend()  
plt.grid(True)  
plt.show()
```



```
In [96]: from IPython.display import Image, display
display(Image("1.2.png"))
```

1.2)

Yes, the estimate of $\frac{\partial L}{\partial \hat{y}}$ is different for different observations :

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y)$$

For $y = 1$: $\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - 1)$

For $y = 2$: $\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - 2)$

At $\hat{y} = 1$:

For $y = 1$, $\frac{\partial L}{\partial \hat{y}} = 2(1 - 1) = 0$

For $y = 2$, $\frac{\partial L}{\partial \hat{y}} = 2(1 - 2) = -2$

✓ Answer: Yes, the estimate for the loss function is different accross different training datapoints. This is because the loss depends on the true value y . For $y = 1$, the curve is

centered at 1 whereas for $y = 2$, the curve is centered at 2 which is why we get different loss values for the same predicted output value.

Yes the choice of observation affects both the loss and the gradient in SGD. Since both depend on the true value y , different datapoints produce different loss values and different gradients for the same prediction. As a result, each observation leads to a different update direction during training.

Recall from multivariable calculus that a function $f(x, y, z)$ has a gradient vector given by

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \\ \frac{\partial f}{\partial z} \end{pmatrix} \quad (1)$$

? 1.3 Since $\hat{y}$ is a function of all the parameters of the network and the input vector x , L is also a function of all these parameters, the inputs, and the true output: $L(x_1, x_2, y, w_1, w_2, w_3, w_4, w_5, w_6, b_1, b_2, b_3)$. For a fixed single observation input and output, x_1, x_2 and y are held constant, making L a function of just 9 parameters for this simple network. Using multivariable chain rule, write out the missing components in terms of partial derivatives.

If you do your work on paper, you don't have to rewrite the whole gradient vector, just write the 5 requested components:

- $\frac{\partial L}{\partial w_1}$
- $\frac{\partial L}{\partial w_2}$
- $\frac{\partial L}{\partial w_5}$
- $\frac{\partial L}{\partial b_2}$
- $\frac{\partial L}{\partial b_3}$

 Answer: (fill in the ??)

$$\nabla L = \begin{pmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \frac{\partial L}{\partial w_3} \\ \frac{\partial L}{\partial w_4} \\ \frac{\partial L}{\partial w_5} \\ \frac{\partial L}{\partial w_6} \\ \frac{\partial L}{\partial b_1} \\ \frac{\partial L}{\partial b_2} \\ \frac{\partial L}{\partial b_3} \end{pmatrix} = \begin{pmatrix} ?? \\ ?? \\ \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_2} \frac{\partial h_2}{\partial w_3} \\ \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_2} \frac{\partial h_2}{\partial w_4} \\ ?? \\ \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_6} \\ \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_1} \frac{\partial h_1}{\partial b_1} \\ ?? \\ ?? \end{pmatrix} \quad (2)$$

```
In [97]: from IPython.display import Image, display
display(Image("1.3.png"))
```

1.3)

hidden layer pre-activations:

$$\begin{aligned} x_1 &= w_1 x_0 + w_2 x_0 + b_1 & \xrightarrow{\text{hidden activations}} & h_1 = f(x_1) \\ x_2 &= w_3 x_1 + w_4 x_2 + b_2 & & h_2 = f(x_2) \end{aligned}$$

output pre-activation:

$$x_3 = h_1 w_5 + h_2 w_6 + b_3 \xrightarrow{\text{output}} \hat{y} = f(x_3)$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial x_3} \cdot \frac{\partial x_3}{\partial h_1} \cdot \frac{\partial h_1}{\partial x_1} \cdot \frac{\partial x_1}{\partial w_1} = 2(\hat{y} - y) \cdot f'(x_3) \cdot w_5 \cdot f'(x_1) \cdot x_0$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial x_3} \cdot \frac{\partial x_3}{\partial h_1} \cdot \frac{\partial h_1}{\partial x_1} \cdot \frac{\partial x_1}{\partial w_2} = 2(\hat{y} - y) \cdot f'(x_3) \cdot w_5 \cdot f'(x_1) \cdot x_0$$

$$\frac{\partial L}{\partial w_5} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial x_3} \cdot \frac{\partial x_3}{\partial w_5} = 2(\hat{y} - y) \cdot f'(x_3) \cdot h_1$$

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial x_3} \cdot \frac{\partial x_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial x_2} \cdot \frac{\partial x_2}{\partial b_2} = 2(\hat{y} - y) \cdot f'(x_3) \cdot w_6 \cdot f'(x_2) \cdot 1$$

$$\frac{\partial L}{\partial b_3} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial x_3} \cdot \frac{\partial x_3}{\partial b_3} = 2(\hat{y} - y) \cdot f'(x_3)$$

Hopefully you can see how many terms (e.g. $\frac{\partial L}{\partial \hat{y}}$) repeat a lot. Backpropagation is going to save us a lot of work by letting us calculate simple partial derivatives just one layer at a time, then combining them one layer at a time. You should also notice 2 facts that will be useful for backpropagation in section 2:

1. Each layer of parameters follows the same pattern of partial derivatives
2. The bias updates and weight updates within the same layer differ by only 1 term

? 1.4 For stochastic gradient descent with just 1 datapoint as input, L depends only on one variable, $\hat{y}$! Write the equation for $\frac{\partial L}{\partial \hat{y}}$ for 1 fixed datapoint, $(\vec{x}, y)$.

In [98]: `from IPython.display import Image, display`
`display(Image("1.4.png"))`

1.4)

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y)$$

? 1.5 As we gave as an example above, the output of the network is $\hat{y} = f(h_1 w_5 + h_2 w_6 + b_3)$. Take the partial derivative of $\hat{y}$ with respect to w_5 . Leave your answer in terms of h_1, h_2, f, f' and the network parameters (weights and biases).

In [99]: `from IPython.display import Image, display`
`display(Image("1.5.png"))`

$$1.5) \quad \frac{\partial \hat{y}}{\partial w_5} = \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_5} = f'(z) \cdot h_1 = f'(h_1 w_5 + h_2 w_6 + b_3) h_1$$

$$\text{note: } z = h_1 w_5 + h_2 w_6 + b_3$$

$$\hat{y} = f(z)$$

? 1.6 Write $\frac{\partial L}{\partial w_5}$ in terms of $f, f', y, \hat{y}, h_1, h_2$, and the network parameters.

In [100]: `from IPython.display import Image, display`
`display(Image("1.6.png"))`

$$1.6) \quad \frac{\partial L}{\partial w_5} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_5} = 2(\hat{y} - y) \cdot f'(z) h_1 = 2(\hat{y} - y) \cdot f'(h_1 w_5 + h_2 w_6 + b_3) h_1$$

? 1.7 Calculate the predicted output, $\hat{y}$, given the following initial parameters:

- $b_1 = b_2 = -1$
- $b_3 = 1/2$

- $w_1 = w_2 = w_3 = w_4 = 1$
- $w_5 = 2$
- $w_6 = -1$

and the inputs $x_1 = 1$ and $x_2 = 0$. Use a sigmoidal activation function, $f(x) = (1 + e^{-x})^{-1}$. Show your work and calculate the intermediates below (those intermediates will be useful for calculating our derivatives!). Given that the true model output for this datapoint should be 1, is this model over or underestimating?

Hint: You should have simple intermediate values and should be able to leave all your answers in a nice simple form until $\hat{y}$, which you may express as a fraction or a decimal.

✓ Answer:

- $h_1 = ?$
- $h_2 = ?$
- $\hat{y} = ?$
- $y = ?$ (this is the true value you should get)

In [101... `from IPython.display import Image, display`
`display(Image("1.7.png"))`

1.7 $\hat{y} = f(z_3) = (h_1 w_5 + h_2 w_6 + b_3)$

hidden layer pre-activations:

$$z_1 = w_1 x_1 + w_2 x_2 + b_1$$

$$z_2 = w_3 x_1 + w_4 x_2 + b_2$$

hidden activations:

$$h_1 = f(z_1)$$

$$h_2 = f(z_2)$$

Intermediate Steps:

$$z_1 = w_1 x_1 + w_2 x_2 + b_1 = 1 \cdot 1 + 1 \cdot 0 + (-1) = 0$$

$$h_1 = f(z_1) = (1 + e^{-0})^{-1} = (1 + 1)^{-1} = \frac{1}{2}$$

$$h_1 w_5 = \frac{1}{2} \cdot 2 = 1$$

$$z_2 = w_3 x_1 + w_4 x_2 + b_2 = 1 \cdot 1 + 1 \cdot 0 + (-1) = 0$$

$$h_2 = f(z_2) = (1 + e^{-0})^{-1} = \frac{1}{2}$$

$$h_2 w_6 = \frac{1}{2} \cdot (-1) = -\frac{1}{2}$$

$$z_3 = h_1 w_5 + h_2 w_6 + b_3 = 1 - \frac{1}{2} + \frac{1}{2} = 1$$

$$\therefore \hat{y} = f(z_3) = (1 + e^{-1})^{-1} = \frac{1}{1 + e^{-1}} = 0.731 \quad \text{and } y = 1$$

$0.731 < 1$ $\therefore$ this model is underestimating

? 1.8 Plug in these intermediate values and your output values to calculate a value for $\frac{\partial L}{\partial w_5}$. Leave your answer as a fraction. Should I increase or decrease w_5 to decrease the loss function? Hopefully you see the utility to keeping the intermediates from the feed-forward pass, and the utility in calculating partial derivatives separately in the backpropagation!

In [102... `from IPython.display import Image, display`


```
display(Image("1.8.png"))
```

1.8)

$$\begin{aligned}\frac{\partial L}{\partial w_5} &= \frac{\partial L}{\partial w_5} = \frac{\partial L}{\partial \hat{q}} \cdot \frac{\partial \hat{q}}{x_3} \cdot \frac{\partial x_3}{\partial w_5} = 2(\hat{q} - y) \cdot f'(x_3) h_1 = 2(\hat{q} - y) \cdot f'(h_1 w_5 + h_2 w_6 + b_3) h_1 \\ &= 2(0.731 - 1) \cdot f'(1) \cdot \frac{1}{2} \\ &= 2(0.731 - 1) \cdot 0.197 \left(\frac{1}{2}\right)\end{aligned}$$

$$\begin{aligned}\text{note: } f'(x) &= f(x) (1 - f(x)) = \frac{1}{1 + e^{-x}} \left(1 - \frac{1}{1 + e^{-x}}\right) = -0.0529 \\ f(1) &= 0.731 (1 - 0.731) = 0.197\end{aligned}$$

Given that we opt a negative slope, we should increase w_5 to decrease the loss function.

Note

Hopefully you can see that this process is **conceptually** very simple but, even with simple numbers and a tiny network, **computationally** annoying. Neural network training turns out to be just *a lot of chain rule*. Make sure you understand this process of backpropagation and could walk through the required calculus for an arbitrary network; given its conceptual simplicity, it's a very exam-able concept.

Section 2 - Constructing a Neural Network Manually

In this section, we're going to construct a neural network "manually" by specifying its architecture and writing functions to compute outputs and train the network.

You're never actually going to train a neural network like this. This question is trying to make sure you understand the internal steps to training and calculating loss over many epochs.

Section 2.1 - Data Prep

Load the dataset

Run the code blocks below to load the hemotoxicity dataset, extract features from the SMILES strings, and scale the features.

```
In [103... !pip install rdkit==2025.9.5
```

```
Requirement already satisfied: rdkit==2025.9.5 in /Users/dianadayekh/anaconda3/envs/clean_cluster/lib/python3.11/site-packages (2025.9.5)
Requirement already satisfied: numpy in /Users/dianadayekh/anaconda3/envs/clean_cluster/lib/python3.11/site-packages (from rdkit==2025.9.5) (2.4.2)
Requirement already satisfied: Pillow in /Users/dianadayekh/anaconda3/envs/clean_cluster/lib/python3.11/site-packages (from rdkit==2025.9.5) (12.1.1)
```

In [104...

```
import numpy as np
import pandas as pd
from rdkit import Chem
from rdkit.Chem import AllChem
from sklearn.preprocessing import StandardScaler

#more datasets here: https://github.com/RekerLab/DeepDelta/tree/main/Dataset
url = "https://raw.githubusercontent.com/RekerLab/DeepDelta/main/Datasets/Be
c = pd.read_csv(url)

# look at the top of this dataset
print(c.head())

# convert SMILES to molecules
mols = [Chem.MolFromSmiles(s) for s in c['SMILES']]

# create binary fingerprints for robust ML models
gen = Chem.AllChem.GetMorganGenerator(radius=2, fpSize=2048)
fps = np.array(gen.GetFingerprint(m) for m in mols)

# create descriptors for ML models with higher interpretability
from rdkit.Chem import Descriptors
descr = Descriptors._descList
calc = [x[1] for x in descr]
d_name = [x[0] for x in descr]

desc = np.array([np.array([f(m) for f in calc]) for m in mols])

# Convert the numpy array to a pandas DataFrame with descriptor names as col
X = pd.DataFrame(desc, columns=d_name)
# extract y values from original dataframe into separate vector
y = c['Y']
# have a look at the top of this dataframe
print(X.head())
print("Shape of the feature data: ", X.shape)

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)
```

	SMILES	Y
0	<chem>CC(C)(C)c1cc(N)c(SCCN)c(NC(=O)c2cccc(C(=O)Nc3c...</chem>	1.146
1	<chem>C0c1cc(OC)c(C(=O)Nc2cc(C(C)(C)C)cc(N)c2SCCN)cc...</chem>	2.161
2	<chem>CC(C)(C)c1cc(N)c(SCCN)c(NC(=O)c2cc(OCCN)cc(C(=...</chem>	2.037
3	<chem>CC(C)(C)c1cc(N)c(SCCNC(N)=[NH2+])c(NC(=O)c2cc(...</chem>	1.987
4	<chem>NCCSc1c(NC(=O)CCCCNC(N)=[NH2+])cc(C(F)(F)F)cc1...</chem>	1.786

	MaxAbsEStateIndex	MaxEStateIndex	MinAbsEStateIndex	MinEStateIndex	\
0	13.468263	13.468263	0.169002	-0.344563	
1	13.926798	13.926798	0.142263	-0.479838	
2	13.814038	13.814038	0.207461	-0.425490	
3	14.119669	14.119669	0.035630	-0.485341	
4	14.127580	14.127580	0.012675	-4.926127	

	qed	SPS	MolWt	HeavyAtomMolWt	ExactMolWt	\
0	0.119644	11.809524	608.878	564.526	608.296717	
1	0.097933	11.695652	668.930	620.546	668.317846	
2	0.091644	11.760870	667.946	618.554	667.333830	
3	0.025868	11.309091	797.093	738.629	796.421054	
4	0.021686	11.354839	918.996	870.612	918.331760	

	NumValenceElectrons	...	fr_sulfide	fr_sulfonamd	fr_sulfone	\
0	226.0	...	2.0	0.0	0.0	
1	250.0	...	2.0	0.0	0.0	
2	250.0	...	2.0	0.0	0.0	
3	298.0	...	2.0	0.0	0.0	
4	338.0	...	2.0	0.0	0.0	

	fr_term_acetylene	fr_tetrazole	fr_thiazole	fr_thiocyan	fr_thiophene	\
0	0.0	0.0	0.0	0.0	0.0	
1	0.0	0.0	0.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	0.0	
3	0.0	0.0	0.0	0.0	0.0	
4	0.0	0.0	0.0	0.0	0.0	

	fr_unbrch_alkane	fr_urea
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	2.0	0.0

[5 rows x 217 columns]

Shape of the feature data: (828, 217)

Data Cleaning

We're going to try to predict hemotoxicity from our input data. You may recall this dataset has no missing features, so we don't need to impute or remove observations. Recall that this dataset does have several zero variance features that won't help our model understand different hemotoxicity values.

? 2.1.1 Remove the 0-variance features from your input dataset using the code below. If we tried to build and train a neural network with all 217 molecular descriptors without

removing the 0-variance features, how large would the input layer have to be? After removing the 0-variance features, how large should our input layer be? If we have 1 hidden layer with 10 neurons, how does this change affect the number of parameters in the model? Assume a fully-connected feed-forward network. Remember the output neuron also has a bias!

```
In [105... # remove 0 variance features
from sklearn.feature_selection import VarianceThreshold
print(X_scaled.shape)
X.dropna(axis=0, how='any', inplace=True) # Dropping rows with NaNs
print(X_scaled.shape)

# another way to identify zero-variance features
selector = VarianceThreshold(threshold=0) # default threshold is 0
X_reduced = selector.fit_transform(X_scaled)
X_reduced.shape
```

```
(828, 217)
```

```
(828, 217)
```

```
Out[105... (828, 190)
```

✓ Answer:

- Size of input layer before 0-variance filtering: 217
- Size of input layer after 0-variance filtering: 190
- Number of model parameters before 0-variance filtering: 2191
- Number of model parameters after 0-variance filtering: 1921

Train-Test-Validation Split

? 2.1.2 Establish a train-test-validation split for these data. Use 80% of the data for training, 10% for test, and 10% for validation. Use `random_state = 0` for each split. *Hint: If you split your data into 90% training and 10% test first, then you just need to figure out how to split the 90% training into training and validation data, where the validation data is 10% of the original dataset.*

Note: Terminology for train/test/validation splits gets confusing. Here, we mean that validation data is used to evaluate how well the training is going, and test data is a held-out set to evaluate model performance after training is finished. You should know that sometimes people flip the definitions for validation and test data.

```
In [106... from sklearn.model_selection import train_test_split
# you're going to have to use the train_test_split function twice
## YOUR CODE HERE
# Step 1: split out test (10%)
X_temp, X_test, y_temp, y_test = train_test_split(
    X_reduced, y, test_size=0.1, random_state=0
)
```

```
# Step 2: split temp into train (80%) and val (10%)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=1/9, random_state=0
)

# double check the sizes of your training and test data
print("Number of observations in the training data: ", X_train.shape[0],
      "\nNumber of observations in the test data: ", X_val.shape[0],
      "\nNumber of observations in the validation data: ", X_val.shape[0])
```

Number of observations in the training data: 662

Number of observations in the test data: 83

Number of observations in the validation data: 83

Section 2.2 - Define the Loss and Activation Functions

Define Loss

We're trying to build a neural network to solve a regression problem. During training and backpropagation, we'll need to repeatedly calculate loss and its derivative, just like in Section 1.

? 2.2.1 Write two python functions to calculate loss and its derivative. Since this is a regression problem, let's use mean-squared error as our loss function. We'll assume a single output from the neural network, $\hat{y}$, per training observation. **When calculating the gradient for a batch of training data, why should our output be a single scalar?**

- Call one function `mse_loss`
- Call its derivative `dL_dydpred`
- Use the test code below to make sure that even when inputting vectors, your functions output a single scalar (not a 1x1 array). This is to ensure compatibility with later code so you can use `np.dot` to multiply this scalar derivative by an array without returning shape multiplication errors.
- *Hint: Both functions should take some true and predicted y-values as input. Both functions should work on arrays of true and predicted values.*

```
In [107... ## YOUR CODE HERE
def mse_loss(y_true, y_pred):
    n = len(y_true)
    return ((y_pred - y_true)**2).mean()

def dL_dydpred(y_true, y_pred):
    n = len(y_true)
    return (2 * (y_pred - y_true)).mean()
```

The test code below should print the following:

```

True y values:
[[0]
 [1]
 [1]
 [1]]
Predicted y values:
[[0]
 [1]
 [1]
 [0]]

-0.5

```

When calculating the gradient for a batch of training data, why should our output be a single scalar? The output should be a single scalar so that it produces an update in a single direction.

```

In [108... # test code
# input example arrays that are 4x1 (column vectors)
example = np.array([0, 1, 1, 1]).reshape(-1,1)
print("True y values: \n", example)
example_pred = np.array([0, 1, 1, 0]).reshape(-1,1)
print("Predicted y values: \n", example_pred)

dL_dy(pred(example),example_pred)
# this output should be -0.5 (given as a scalar)

```

```

True y values:
[[0]
 [1]
 [1]
 [1]]
Predicted y values:
[[0]
 [1]
 [1]
 [0]]

```

```

Out[108... np.float64(-0.5)

```

Define Activation Function

Earlier, we wrote all our backpropagation functions in terms of the activation function, f , and its derivative, f' .

? 2.2.2 Still using the sigmoidal activation function, write two python functions that will evaluate the activation function and its derivative. Call these functions `sigmoid(x)` and `sigmoid_derivative(x)`.

You can run the second code block to check that you produce the correct result for the sigmoid function.

In [109... `## ? YOUR CODE HERE`

```

# Activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of the activation function
def sigmoid_derivative(x):
    s = sigmoid(x)
    return s * (1 - s)

```

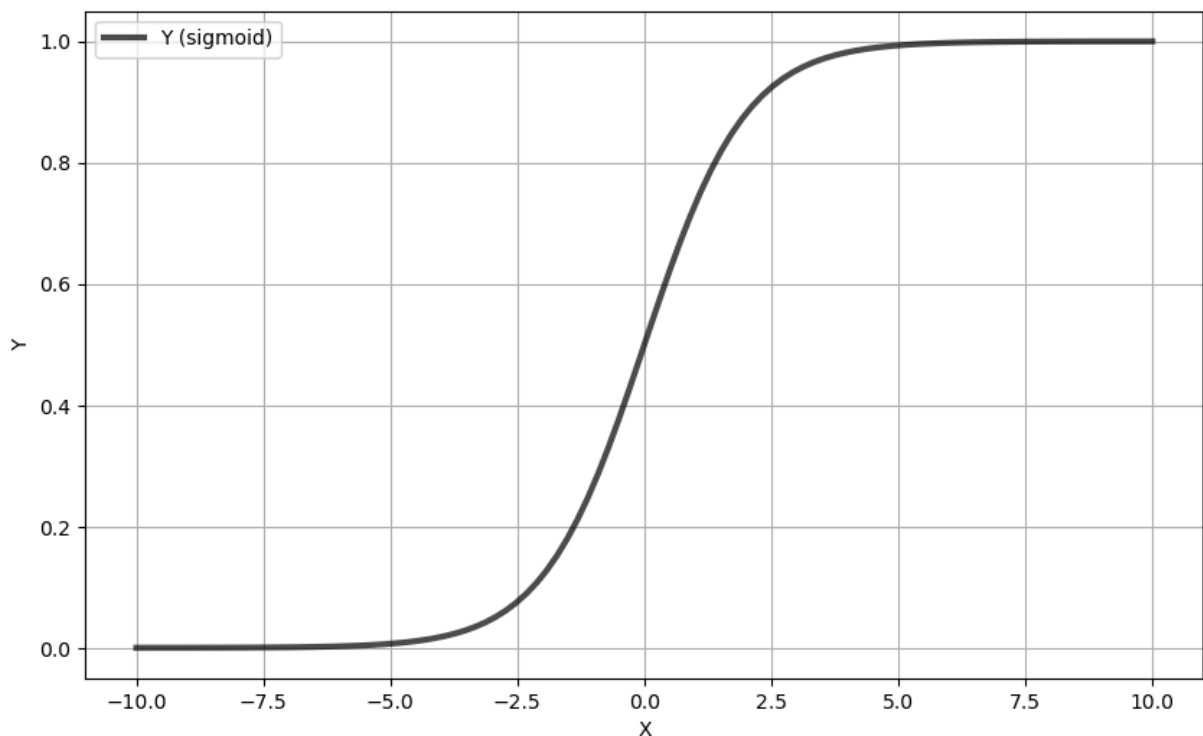
In [110... `# Reminder of what the sigmoid function should look like`

```

import matplotlib.pyplot as plt

example_x = np.linspace(-10, 10, 100)
plt.figure(figsize=(10, 6))
plt.plot(example_x, sigmoid(example_x), label='Y (sigmoid)', linewidth=3, color='red')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.grid()
plt.show()

```



? 2.2.3 Earlier, our toy neural network used the same activation function for the hidden layer and for the output layer. Should we apply the sigmoid activation function to our output layer in this neural network? Why or why not? *Hint: The code below makes a plot to remind you of the range of the hemotoxicity output values we're trying to model.*

In [111... `import matplotlib.pyplot as plt`

```

fig, (ax_box, ax_hist) = plt.subplots(nrows=2, sharex=True,

```

```

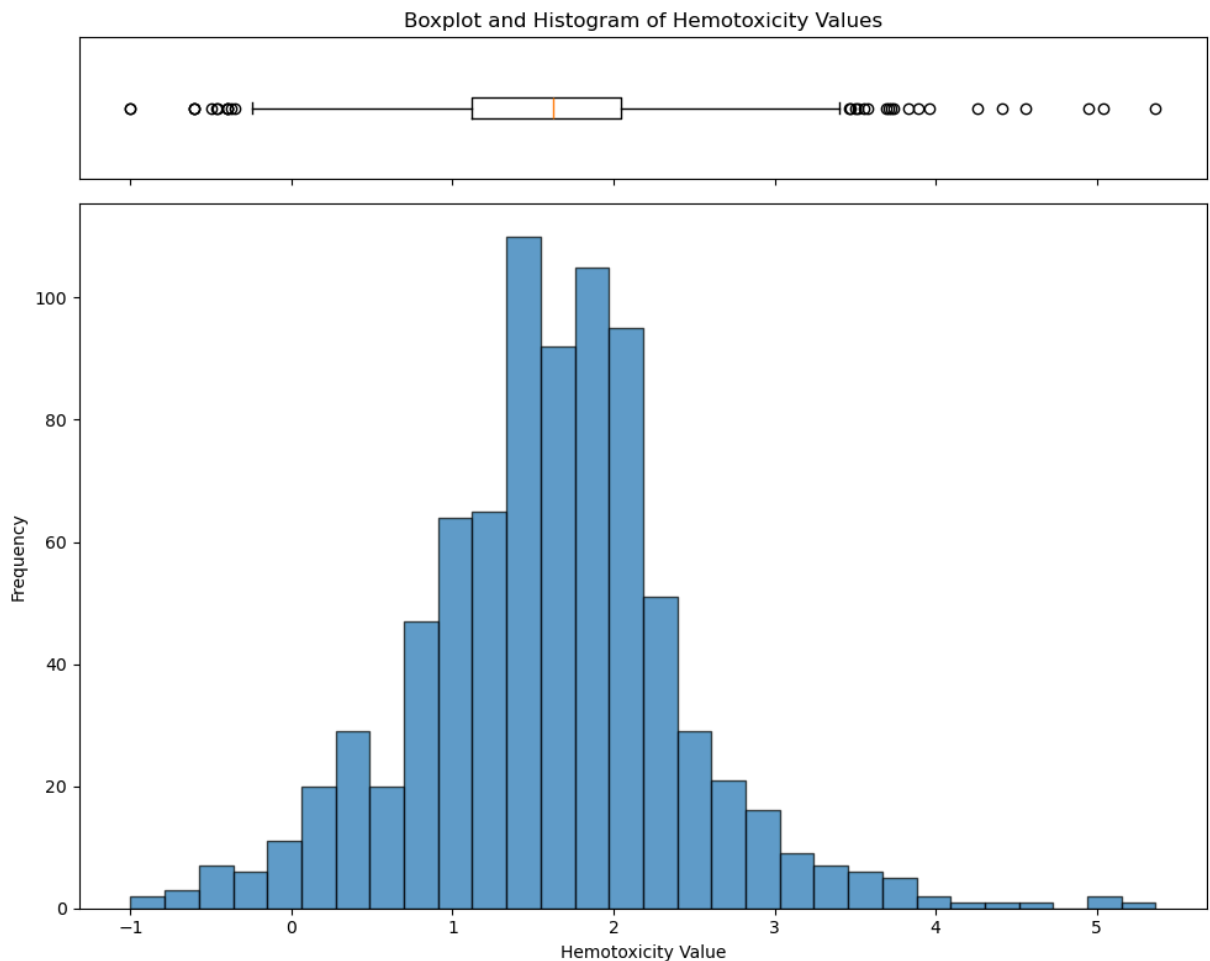
gridspec_kw={"height_ratios": [1, 5]},
figsize=(10, 8))

# Box plot on the first subplot
ax_box.boxplot(y, vert=False)
ax_box.set_title('Boxplot and Histogram of Hemotoxicity Values')
ax_box.set_yticks([]) # Remove y-ticks for the boxplot

# Histogram on the second subplot
ax_hist.hist(y, bins=30, edgecolor='black', alpha=0.7)
ax_hist.set_xlabel('Hemotoxicity Value')
ax_hist.set_ylabel('Frequency')

plt.tight_layout()
plt.show()

```



✓ Answer: No because the range of output values we would get with the sigmoid function is between 0 and 1 and the histogram indicates that we need a range of values between 0 and 200.

Section 2.3 - Defining the Neural Network

Now we're going to instantiate a `class` for our Neural Network and use that to specify a (somewhat) flexible architecture.

What's a Class? (Some Python Background)

Classes are a type of objects in Python. Normally in `scikit-learn` when you're fitting a model, you do something like this:

```
from sklearn.tree import DecisionTreeRegressor

model = DecisionTreeRegressor(random_state=12, max_depth=4)
model.fit(X, y)
```

In this case, I've fit a depth-limited decision tree to the whole dataset contained in `X` and it's outputs, `y`. But what was `model` before I called `model.fit` to fit it? It was just an instance of the `DecisionTreeRegressor` **class** with some specified parameters: a `random_state` and the constraint that the tree should only have 4 levels, plus some other defaults, as found in the [documentation](#).

In the code above, `model` is not a function, or a list of parameters, or a dataframe. `model` has a list of parameters associated with it (a bunch of defaults, plus a specified `random_state` and `max_depth`), but it also does "stuff" like a function would, and stores some interesting results of things it does. **The things a class can do are usually called methods and the information about itself that a class might store are called attributes.**

In the example above, we call `fit`, a method associated with the `DecisionTreeRegressor` class. This is akin to applying a function to both `model` and our `X` data, and uses the parameters stored in `model` to fit a decision tree. Example **attributes** of a decision tree include feature importance information and some other outputs that are produced whenever you call the method `fit`.

The first thing we did in the code above was to create an empty "instance" of the `DecisionTreeRegressor` class and call it `model`. That initialization is called "instantiation." Every class has a default method that runs as soon as you instantiate it; that default method is always called `__init__`. As an example, here is a class called `NeuralNetwork` that instantiates by storing a bunch of parameters, some of which have default values:

```
In [112... import numpy as np

# define a class!
class NeuralNetwork:
    def __init__(self, input_size, output_size, learning_rate, hidden_size = 1
        # store parameters I want to access later in self
        self.input_size = input_size
        self.output_size = output_size
        self.learning_rate = learning_rate
        self.hidden_size = hidden_size
        self.epochs = epochs
```

```

def print_about_me(self):
    # this is an example of a method, which is like a function that can act
    # in this case, the method just takes the classes "self" as input

    # intermediate values I don't want to store as a part of the class can e
    # just like with any other function
    total_num_neurons = self.hidden_size + self.output_size
    print("Input size of ", self.input_size,
          "\nTotal Number of neurons: ", total_num_neurons)

# create an instance of this class!
nn = NeuralNetwork(input_size = 2,
                    output_size = 1,
                    learning_rate = 0.01,
                    hidden_size = 2)

# call the method print_about_me to tell me some things about this network
nn.print_about_me()

```

Input size of 2

Total Number of neurons: 3

Methods always take `self` as one of their inputs, to make sure the method has access to all the parameters, data, and other functions stored in the class. Methods can ...

- create and store new variables in the class (as a part of `self`)
- update old variables in the class (useful for backpropagation!)
- create temporary variables within the function that get stored as belonging to the class
- take other inputs from outside the class (e.g. `DecisionTreeRegressor.fit` takes X and y data to fit the model).
- return values
- use/call other previously established methods in the class

Takeaway: Classes are a flexible way to store associated data and functions together. You've been using them the whole time! Now we're going to build our own `class` that defines a particular neural network architecture.

External resources on using classes in Python:

- [Introduction to Python Classes](#) on TowardsDataScience.com.
- [Python tutorial on Classes](#)

Ok, back to Section 2.3

We'll copy/paste code from one question to the next to build up the class one method (function) at a time, translating the written instructions here to code instructions for the neural network. For this section, **use the skeleton code** and answer the marked questions throughout.

? 2.3.1 Complete the code to instantiate the `NeuralNetwork` class. This neural network should have a single hidden layer of adjustable size and a single output.

- The instantiation should use `np.random.randn()` to randomly guess at weights for the whole network. Variables for the vectors/matrices of weights have been named for you.
- Initialize all the biases to 0.
- *To check that you're correctly instantiating the neural network class, you can use the `print_check` method from the skeleton code to print various intermediates or their shapes to make sure you understand what structure you've specified.*
- ? 2.3.2 Describe `self.weights_input_to_hidden`. What is the shape of this numpy array? What are the initial values established when the `NeuralNetwork` class is instantiated?
 - ✓ Answer: Each input feature connects to every neuron in the hidden layer, so the weight matrix has shape (190, 5), meaning 190 rows (input features) and 5 columns (hidden neurons). The initial values are randomly generated using `np.random.randn()`.
- ? 2.3.3 How many total biases did you initialize as 0s in the example with a single hidden layer of 5 neurons and a single output neuron?
 - ✓ Answer: There are 5 biases for the hidden layer (one per hidden neuron) and 1 bias for the output neuron, for a total of 6 biases initialized to 0.

With the `print_check` method left as given in the skeleton code, the test code should output:

```
(190, 5)
[[0. 0. 0. 0. 0.]
```

```
In [113]: # skeleton code, fill in as necessary

# define the NeuralNetwork class
class NeuralNetwork:
    def __init__(self, input_size, learning_rate, hidden_size, epochs = 1000):
        # set seed for reproducibility
        np.random.seed(12)
        # initialize weights connecting the input and hidden layers
        self.weights_input_to_hidden = np.random.randn(input_size, hidden_size)
        # initialize biases for the hidden layer of neurons
        self.bias_hidden = np.zeros((1, hidden_size))

        # also initialize biases and weights on output layer
        # use the initialized values on the first layer as an example
        self.weights_hidden_to_output = np.random.randn(hidden_size, 1) # use np.
        self.bias_output = np.zeros((1, 1)) # use np.zeros to establish

        # store other user-defined inputs in the instantiated class for access
        self.learning_rate = learning_rate
        self.epochs = epochs
```

```
def print_check(self):
    # this is a method to call to make sure you're correctly making intermediates
    # rewrite this method to print whatever you're interested in checking
    print(self.weights_input_to_hidden.shape)
    print(self.bias_hidden)

## Now try instantiating a neuralnetwork class!
nn = NeuralNetwork(input_size=190, hidden_size=5, learning_rate = 0.01)

# call the print_check method to check intermediates are properly instantiated
# this is a troubleshooting check for you! Change the print_check method to
nn.print_check()
```

```
(190, 5)
[[0. 0. 0. 0. 0.]
```

? 2.3.4 Define a method to do a forward pass.

- This method should take the input feature matrix as input and produce an output hemotoxicity estimate, given the current model weights!
- This forward pass should work on an arbitrary number of observations, not just 1 at a time.
- Make sure to store inputs and outputs for backpropagation (see skeleton code, all variable names have been provided for you).
- The hidden layer of neurons should use the sigmoidal activation function
- The output layer of neurons should not have an activation function
- ★ 2.3.5 Bonus - What is `self.hidden_input` ? Why are we storing it? *Hint: Think back to section 1 and where this gets used in backpropagation. You can also answer this question after looking at the provided backpropagation code.*
 - 🍓 Answer: `self.hidden_input` is the pre-activation input to the hidden layer, computed as the weighted sum of the input features plus the bias. It is stored because it is needed during backpropagation to compute the derivative of the activation function.
- ? 2.3.6 With 2 inputs, 5 neurons in a single hidden layer, 1 output neuron, and 3 training observations, how many output values do we store in `self.output` ?
 - ✅ Answer: 3

The test code should output the following:

```
Model Input observations:
[[0 1]
 [1 1]
 [0 0]]
```

```
Model Output:
array([[1.54144736],
       [1.80303923],
       [0.54888888]])
```

```

In [114... # copy paste your complete initialization method from above!
# define the NeuralNetwork class
class NeuralNetwork:
    def __init__(self, input_size, learning_rate, hidden_size, epochs = 1000):
        ##### copy paste your code from above here
        # set seed for reproducibility
        np.random.seed(12)
        # initialize weights connecting the input and hidden layers
        self.weights_input_to_hidden = np.random.randn(input_size, hidden_size)
        # initialize biases for the hidden layer of neurons
        self.bias_hidden = np.zeros((1, hidden_size))

        # also initialize biases and weights on output layer
        # use the initialized values on the first layer as an example
        self.weights_hidden_to_output = np.random.randn(hidden_size,1) # use np.
        self.bias_output = np.zeros((1,1)) # use np.zeros to establish

        # store other user-defined inputs in the instantiated class for access
        self.learning_rate = learning_rate
        self.epochs = epochs

    def print_check(self):
        # this is a method to call to make sure you're correctly making intermed
        # rewrite this method to print whatever you're interested in checking
        print(self.weights_input_to_hidden.shape)
        print(self.bias_hidden)
        print(self.weights_hidden_to_output.shape)
        print(self.bias_output)

    def forward(self, X):
        # do a forward pass using variables instantiated above
        # remember to store the inputs and outputs for use in backpropagation
        self.hidden_input = np.dot(X, self.weights_input_to_hidden) + self.bias_hidden
        self.hidden_output = sigmoid(self.hidden_input) #apply activation func

        # calculate outputs from the hidden layer outputs and the weights
        # do not apply any activation function to this output!
        self.output = np.dot(self.hidden_output, self.weights_hidden_to_output)
        return self.output

## Test code!
# first create an instance of the NeuralNetwork class
nn = NeuralNetwork(input_size=2, hidden_size=5, learning_rate = 0.01)

# try a forward pass on a tiny example input, print inputs and prediction
example_input = np.array([[0,1], [1,1], [0,0]])
print("Model Input observations:\n", example_input)
print("\nModel Output:")
nn.forward(example_input)
# this output should be a column vector with 3 predictions!

```

Model Input observations:

```
[[0 1]
 [1 1]
 [0 0]]
```

Model Output:

```
Out[114...] array([[1.54144736],
 [1.80303923],
 [0.54888888]])
```

? 2.3.7 Inspect the given method that does a "backward" pass of backpropagation and **complete the last four lines of updating the weights and biases of the model.**

- This function calculates the gradient and update the model weights and biases according to the calculated gradient and the learning rate, performing one "backward pass" of backpropagation.
- It uses all the training data passed to the `backward` method to estimate the gradient and do the backpropagation pass. (this could be multiple observations in a batch or minibatch, or a single observation for stochastic gradient descent)
- The `--` operator is used to subtract and update - it subtracts the right side from the left side operand and then assigns the result to left operand.
- *Re-estimating the parameters looks suspiciously like Euler's method from Calculus I, or solving differential equations numerically!*

The test code provided should output the following (for the `print_check` method in the code block below):

Model Input observations:

```
[[0 1]
 [1 1]
 [0 0]]
```

Model Output:

```
[[ 0.18508331]
 [ 0.37605972]
 [-0.39078925]]
```

Model True outputs:

```
[[0]
 [1]
 [1]]
```

Weights from the input to hidden layer:

```
[[ 0.47298583 -0.68142588]
 [ 0.2424395 -1.70073563]]
```

Biases on the hidden layer:

```
[[0. 0.]]
```

Weights from the input to hidden layer:

```
[[ 0.47240854 -0.68200317]
 [ 0.24204268 -1.70113245]]
```

Biases on the hidden layer:
[[0.00198654 0.00198654]]

```
In [115... # copy paste your code from above to fill in the method here!

# define the NeuralNetwork class
class NeuralNetwork:
    def __init__(self, input_size, learning_rate, hidden_size, epochs = 1000):
        ##### copy paste your code from above here
        # set seed for reproducibility
        np.random.seed(12)
        # initialize weights connecting the input and hidden layers
        self.weights_input_to_hidden = np.random.randn(input_size, hidden_size)
        # initialize biases for the hidden layer of neurons
        self.bias_hidden = np.zeros((1, hidden_size))

        # also initialize biases and weights on output layer
        # use the initialized values on the first layer as an example
        self.weights_hidden_to_output = np.random.randn(hidden_size,1) # use np.
        self.bias_output = np.zeros((1,1)) # use np.zeros to establish

        # store other user-defined inputs in the instantiated class for access
        self.learning_rate = learning_rate
        self.epochs = epochs

    def print_check(self):
        # this is a method to call to make sure you're correctly making inte
        # rewrite this method to print whatever you're interested in checkin
        print("Weights from the input to hidden layer: \n", self.weights_inp
        print("Biases on the hidden layer: \n", self.bias_hidden)

    def forward(self, X):
        # do a forward pass using variables instantiated above
        # remember to store the inputs and outputs for use in backpropagation
        self.hidden_input = np.dot(X, self.weights_input_to_hidden) + self.bia
        self.hidden_output = sigmoid(self.hidden_input) #apply activation func

        # calculate outputs from the hidden layer outputs and the weights
        # do not apply any activation function to this output!
        self.output = np.dot(self.hidden_output, self.weights_hidden_to_output)
        return self.output

    def backward(self,X,y,y_pred):
        # backward pass (gradient descent using backpropagation)
        # derivative of loss wrt ypred, this should be a scalar if your function
        loss_der = dL_dypred(y, y_pred)

        # Gradients for weights and biases between hidden layer and output layer
        # no activation function on the output!
        w_hidden_to_output_grad = np.sum(np.dot(self.hidden_output, loss_der),
                                           axis = 0, keepdims=True).T
        bias_output_grad = loss_der # no activation function to apply here

        # Gradients for weights and biases between input and hidden
        hidden_error = np.dot(sigmoid_derivative(self.hidden_input), np.dot(loss
```

```

w_input_to_hidden_grad = np.dot(X.T, hidden_error)
bias_hidden_grad = np.sum(hidden_error, axis=0, keepdims=True)

# Update weights and biases
self.weights_hidden_to_output -= self.learning_rate * w_hidden_to_output
self.bias_output -= self.learning_rate * bias_output_grad ##
self.weights_input_to_hidden -= self.learning_rate * w_input_to_hidden_grad
self.bias_hidden -= self.learning_rate * bias_hidden_grad ##

# testing this - first create an instance of the NeuralNetwork class
nn = NeuralNetwork(input_size=2, hidden_size=2, learning_rate = 0.01)

# we need to do a forward pass on our data before we can do a backward pass!
example_input = np.array([[0,1], [1,1], [0,0]])
print("Model Input observations:\n", example_input)
example_predictions = nn.forward(example_input)
print("\nModel Output: \n", example_predictions)
# this output should be a column vector with 3 predictions!

## try a backpropagation pass!
# First we'll need some true output data
example_true_output = np.array([0,1,1])
print("Model True outputs:\n", example_true_output, "\n")

# now we can try backpropagation using our predictions and true
nn.print_check()
nn.backward(example_input, example_true_output, example_predictions)
nn.print_check()

```

Model Input observations:

```
[[0 1]
 [1 1]
 [0 0]]
```

Model Output:

```
[[ 0.18508331]
 [ 0.37605972]
 [-0.39078925]]
```

Model True outputs:

```
[0 1 1]
```

Weights from the input to hidden layer:

```
[[ 0.47298583 -0.68142588]
 [ 0.2424395  -1.70073563]]
```

Biases on the hidden layer:

```
[[0. 0.]]
```

Weights from the input to hidden layer:

```
[[ 0.47356312 -0.68084859]
 [ 0.24283631 -1.70033882]]
```

Biases on the hidden layer:

```
[[-0.00198654 -0.00198654]]
```

? 2.3.8 Complete the code to build a method to train the neural network.

- We'll follow the typical `scikit-learn` vocabulary and call this `train`.
- This method should take the training data feature matrix, `X`, and the training data true outputs, `y`, as inputs
 - Inside the method, this `X` is not the global variable, but just whatever matrix is passed to the method.
- Use ALL the provided training data as a batch to complete one epoch, making predictions in a forward pass and then updating the gradient in a backwards pass.

```
In [116... # copy paste your code from above to fill in the methods here!

# define the NeuralNetwork class
class NeuralNetwork:
    def __init__(self, input_size, learning_rate, hidden_size, epochs = 1000):
        ##### copy paste your code from above here
        # set seed for reproducibility
        np.random.seed(12)
        # initialize weights connecting the input and hidden layers
        self.weights_input_to_hidden = np.random.randn(input_size, hidden_size)
        # initialize biases for the hidden layer of neurons
        self.bias_hidden = np.zeros((1, hidden_size))

        # also initialize biases and weights on output layer
        # use the initialized values on the first layer as an example
        self.weights_hidden_to_output = np.random.randn(hidden_size, 1) # use np.
        self.bias_output = np.zeros((1, 1)) # use np.zeros to establish

        # store other user-defined inputs in the instantiated class for access later
        self.learning_rate = learning_rate
        self.epochs = epochs

    def print_check(self):
        # this is a method to call to make sure you're correctly making internal checks
        # rewrite this method to print whatever you're interested in checking
        print(self.weights_input_to_hidden.shape)
        print(self.bias_hidden)

    def forward(self, X):
        # do a forward pass using variables instantiated above
        # remember to store the inputs and outputs for use in backpropagation
        self.hidden_input = np.dot(X, self.weights_input_to_hidden) + self.bias_hidden
        self.hidden_output = sigmoid(self.hidden_input) #apply activation function

        # calculate outputs from the hidden layer outputs and the weights
        # do not apply any activation function to this output!
        self.output = np.dot(self.hidden_output, self.weights_hidden_to_output)
        return self.output

    def backward(self, X, y, y_pred):
        # backward pass (gradient descent using backpropagation)
        # derivative of loss wrt ypred, this should be a scalar if your function is a scalar
        loss_der = dL_dy(pred(y, y_pred))

        # Gradients for weights and biases between hidden layer and output layer
```

```

# no activation function on the output!
w_hidden_to_output_grad = np.sum(np.dot(self.hidden_output, loss_der),
                                   axis = 0, keepdims=True).T
bias_output_grad = loss_der # no activation function to apply here

# Gradients for weights and biases between input and hidden
hidden_error = np.dot(sigmoid_derivative(self.hidden_input), np.dot(loss_der,
w_input_to_hidden_grad = np.dot(X.T, hidden_error)
bias_hidden_grad = np.sum(hidden_error, axis=0, keepdims=True)

# Update weights and biases
self.weights_hidden_to_output -= self.learning_rate * w_hidden_to_output_grad
self.bias_output -= self.learning_rate * bias_output_grad ##
self.weights_input_to_hidden -= self.learning_rate * w_input_to_hidden_grad
self.bias_hidden -= self.learning_rate * bias_hidden_grad ##

def train(self, X, y):
    # method for training the neural network, given some external training data
    for epoch in range(self.epochs):
        # DO A FORWARD PASS TO MAKE PREDICTIONS
        y_pred = self.forward(X) ## YOUR CODE HERE

        ## DO A BACKWARD PASS
        self.backward(X, y, y_pred)

        # print the loss every 100 epochs
        if epoch % 100 == 0:
            loss = mse_loss(y, y_pred)
            print(f"Epoch {epoch}, Loss: {loss}")

# Initialize and train the neural network USING THESE PARAMETERS
nn = NeuralNetwork(input_size=190, hidden_size=25, learning_rate = 0.000001,
nn.train(X_train, y_train.to_numpy().reshape(-1,1))

# this should print a bunch of epoch-by-epoch losses, AND train your model

```

```

Epoch 0, Loss: 27.341375531356693
Epoch 100, Loss: 12.067150321906166
Epoch 200, Loss: 8.866471389876516
Epoch 300, Loss: 8.060646159414024
Epoch 400, Loss: 7.817488727098804
Epoch 500, Loss: 7.732568772740637
Epoch 600, Loss: 7.700143202127979
Epoch 700, Loss: 7.687194465975113
Epoch 800, Loss: 7.681917832326958
Epoch 900, Loss: 7.679748816487792

```

Section 2.4 - Making Some Predictions

Now we're going to apply our final NeuralNetwork class to our hemotoxicity data.

? 2.4 Use your simple neural network to make predictions on your test data. Report the RMSE of your test data. Based on the losses reported per epoch above, do you expect

your predictions to be good? Were your predictions good? (The LASSO and decision tree models we built a couple weeks ago using this dataset had RMSE values between 0.7 and 0.8).

```
In [117... from sklearn.metrics import root_mean_squared_error

# predict on new data
y_predicted = nn.forward(X_test)
# check out the RMSE
rmse = root_mean_squared_error(y_test, y_predicted)
print("RMSE:", rmse)
```

RMSE: 2.958186508093634

✓ Answer: Based on the loss values decreasing over epochs, the model is learning, so one might initially expect the predictions to be reasonably good. However, the loss plateaus around ~7.67, indicating that the model stops improving significantly and is not fitting the data well.

The RMSE on the test set is approximately 2.96, which is much higher than the RMSE values of 0.7–0.8 achieved by the LASSO and decision tree models. This shows that the neural network's predictions are not good.

Section 3 - Constructing a Neural Network using SciKitLearn

Now we're going to use scikit-learn to implement a neural network quickly. We'll use `MLPRegressor` and its accompany `partial_fit` method for training, plotting the loss over every epoch. The `partial_fit` method performs one iteration of model updates over the given data. If we provide `partial_fit` with a full training data set, it will perform 1 epoch.

? 3.1 Write code to initialize the multi-layer perceptron regressor and plot loss (MSE) as a function of epoch.

- Specify 2 hidden layers, the first with 50 neurons and the second with 5 neurons.
- Use a `relu` activation function and start with an initial (and constant) learning rate of 0.001.
 - *We haven't talked about it yet, but you may notice options for a non-constant learning rate. Adaptive learning rates help your model learn faster at first and then make smaller adjustments later in the learning process.*
- Use stochastic gradient descent as the `solver` for your model. Notice that this gives you a default batch size, use the default for now.
- Usually the `max_iter` parameter determines the number of training epochs to run, but since we're going to record and plot the loss on a per-epoch basis, we'll do that in a `for` loop ourselves, so set `max_iter=1`.

- Set `warm_start=True` . This will let us use `partial_fit` in the for loop to iterate manually through epochs.
- Set `random_state=12`
- Run your for loop for 2000 epochs and plot MSE (loss) as a function of epoch.

```
In [118... from sklearn.neural_network import MLPRegressor

mlp = MLPRegressor(
    hidden_layer_sizes=(50, 5),
    activation='relu',
    learning_rate_init=0.001,
    solver='sgd',
    max_iter=1,
    warm_start=True,
    random_state=12
) ## YOUR CODE HERE

# Number of epochs
epochs = 2000

# create empty arrays to store training and testing losses for each epoch
train_losses = []
val_losses = []

# Training loop
for epoch in range(epochs):
    # Train on the training set using mlp.partial_fit
    mlp.partial_fit(X_train, y_train) ## YOUR CODE HERE

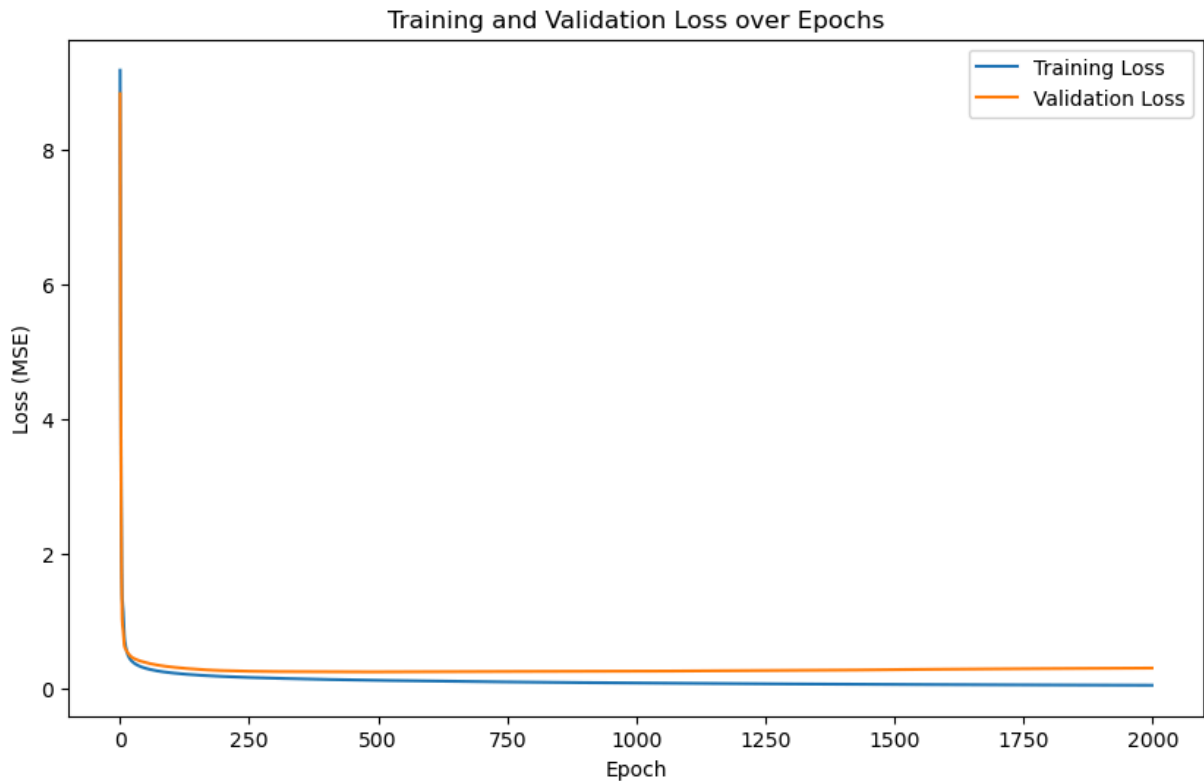
    # Compute training loss (MSE) for this epoch
    # using mlp.predict and the vector of true y training values
    train_loss = np.mean((mlp.predict(X_train) - y_train) ** 2) ## YOUR CODE
    train_losses.append(train_loss)

    # Compute validation loss (MSE) using mlp.predict and the vector of true
    val_loss = np.mean((mlp.predict(X_val) - y_val) ** 2) ## YOUR CODE HERE
    val_losses.append(val_loss)

    # Print losses every 100 epochs
    if epoch % 100 == 0:
        print(f"Epoch {epoch}, Train Loss: {train_loss}, Validation Loss: {val_loss}")

# Plot the losses!
plt.figure(figsize=(10, 6))
plt.plot(train_losses, label="Training Loss")
plt.plot(val_losses, label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.title("Training and Validation Loss over Epochs")
plt.legend()
plt.show()
```

Epoch 0, Train Loss: 9.17372637714911, Validation Loss: 8.8266894418527
Epoch 100, Train Loss: 0.238631372518709, Validation Loss: 0.32525308582139767
Epoch 200, Train Loss: 0.1852087999410129, Validation Loss: 0.2723062000218847
Epoch 300, Train Loss: 0.1591975957804999, Validation Loss: 0.2583074627403549
Epoch 400, Train Loss: 0.1411872728296923, Validation Loss: 0.2544944880852058
Epoch 500, Train Loss: 0.12808575465968666, Validation Loss: 0.2534616060914975
Epoch 600, Train Loss: 0.11745392533331196, Validation Loss: 0.2557107915835387
Epoch 700, Train Loss: 0.10831565693538088, Validation Loss: 0.2585911547133099
Epoch 800, Train Loss: 0.1004804251056352, Validation Loss: 0.2604744443990161
Epoch 900, Train Loss: 0.09389467806731426, Validation Loss: 0.26225747300555446
Epoch 1000, Train Loss: 0.08795951248846585, Validation Loss: 0.2644035576048203
Epoch 1100, Train Loss: 0.08276912698754861, Validation Loss: 0.2659354468154604
Epoch 1200, Train Loss: 0.07819053224177516, Validation Loss: 0.26949760842590736
Epoch 1300, Train Loss: 0.07414432239467983, Validation Loss: 0.27400520207106305
Epoch 1400, Train Loss: 0.07055588113641438, Validation Loss: 0.27958477039079516
Epoch 1500, Train Loss: 0.0673531113407622, Validation Loss: 0.2866245519764875
Epoch 1600, Train Loss: 0.0644892581817517, Validation Loss: 0.29261959322025305
Epoch 1700, Train Loss: 0.061875002219784536, Validation Loss: 0.29781511466643723
Epoch 1800, Train Loss: 0.05962004666533248, Validation Loss: 0.3022672828366151
Epoch 1900, Train Loss: 0.05742557283106755, Validation Loss: 0.30612534062599683



? 3.2 Adjust the learning rate so that your network learns "too fast." Briefly explain your output plots. Thinking about stochastic gradient descent on the loss function landscape, what happened here?

```
In [119... # copy/paste necessary code from above
from sklearn.neural_network import MLPRegressor

mlp = MLPRegressor(
    hidden_layer_sizes=(50, 5),
    activation='relu',
    learning_rate_init=0.04,
    solver='sgd',
    max_iter=1,
    warm_start=True,
    random_state=12
) ## YOUR CODE HERE

# Number of epochs
epochs = 2000

# create empty arrays to store training and testing losses for each epoch
train_losses = []
val_losses = []

# Training loop
for epoch in range(epochs):
    # Train on the training set using mlp.partial_fit
    mlp.partial_fit(X_train, y_train) ## YOUR CODE HERE
```

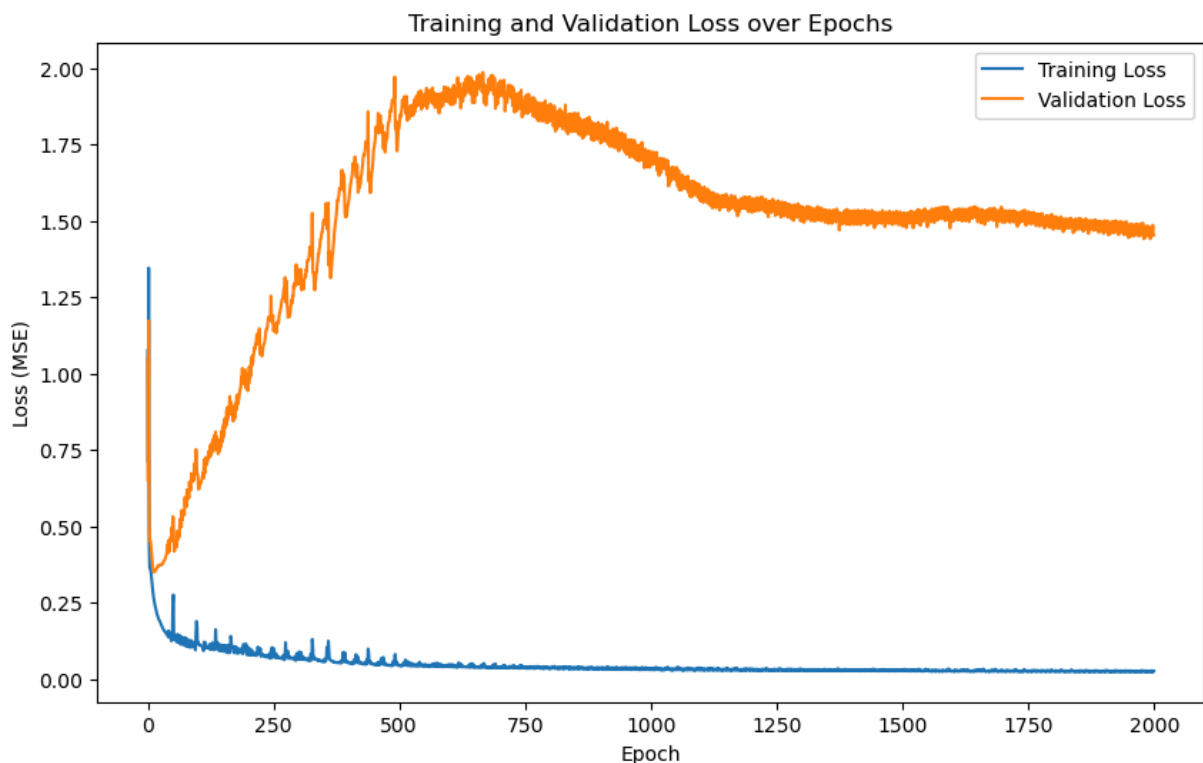
```
# Compute training loss (MSE) for this epoch
# using mlp.predict and the vector of true y training values
train_loss = np.mean((mlp.predict(X_train) - y_train) ** 2)## YOUR CODE
train_losses.append(train_loss)

# Compute validation loss (MSE) using mlp.predict and the vector of true
val_loss = np.mean((mlp.predict(X_val) - y_val) ** 2)## YOUR CODE HERE
val_losses.append(val_loss)

# Print losses every 100 epochs
if epoch % 100 == 0:
    print(f"Epoch {epoch}, Train Loss: {train_loss}, Validation Loss: {val_loss}")

# Plot the losses!
plt.figure(figsize=(10, 6))
plt.plot(train_losses, label="Training Loss")
plt.plot(val_losses, label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.title("Training and Validation Loss over Epochs")
plt.legend()
plt.show()
```

Epoch 0, Train Loss: 1.075219408844914, Validation Loss: 1.0478695439307235
Epoch 100, Train Loss: 0.11884593153465783, Validation Loss: 0.6557798334082532
Epoch 200, Train Loss: 0.10302556805729284, Validation Loss: 0.9985562414001863
Epoch 300, Train Loss: 0.06594407920970413, Validation Loss: 1.2940914339026834
Epoch 400, Train Loss: 0.05306901436097596, Validation Loss: 1.571329138342428
Epoch 500, Train Loss: 0.04489134353627932, Validation Loss: 1.8116708520280242
Epoch 600, Train Loss: 0.041978877839387314, Validation Loss: 1.9276983348105219
Epoch 700, Train Loss: 0.038937447434485295, Validation Loss: 1.944533277583139
Epoch 800, Train Loss: 0.03408721414564327, Validation Loss: 1.8555075068372324
Epoch 900, Train Loss: 0.03311002899475256, Validation Loss: 1.7544552845142405
Epoch 1000, Train Loss: 0.03356871378493617, Validation Loss: 1.7198711045610517
Epoch 1100, Train Loss: 0.028518476861929414, Validation Loss: 1.5639502692587037
Epoch 1200, Train Loss: 0.03173357907468298, Validation Loss: 1.5632852715554633
Epoch 1300, Train Loss: 0.029374075160914868, Validation Loss: 1.5281500532233991
Epoch 1400, Train Loss: 0.026810996436597904, Validation Loss: 1.5274627182204643
Epoch 1500, Train Loss: 0.025896485996362982, Validation Loss: 1.4825557660033224
Epoch 1600, Train Loss: 0.026119913194586394, Validation Loss: 1.5264683958448695
Epoch 1700, Train Loss: 0.025110574242765728, Validation Loss: 1.5151792826009087
Epoch 1800, Train Loss: 0.02434169927541725, Validation Loss: 1.5041722661260266
Epoch 1900, Train Loss: 0.02295183901956104, Validation Loss: 1.4838237117786455



✓ Answer: Increasing the learning rate makes the model unstable because the weight updates become too large. Instead of gradually approaching the minimum of the loss function, the model overshoots it and oscillates around it. This leads to noisy and fluctuating loss curves, especially for the validation set. In the plots, this appears as irregular, non-smooth behavior compared to the smoother convergence seen with a smaller learning rate. From the perspective of stochastic gradient descent, the step size is too large, preventing the model from settling into a minimum of the loss landscape.

★ 3.3 Bonus - Using your original learning rate, try changing the activation function to a sigmoid. How does this affect your model training? How does it affect the model's error on test data after 2000 epochs?

```
In [120... from sklearn.neural_network import MLPRegressor

mlp = MLPRegressor(
    hidden_layer_sizes=(50, 5),
    activation='logistic',
    learning_rate_init=0.001,
    solver='sgd',
    max_iter=1,
    warm_start=True,
    random_state=12
) ## YOUR CODE HERE

# Number of epochs
epochs = 2000

# create empty arrays to store training and testing losses for each epoch
```

```

train_losses = []
val_losses = []

# Training loop
for epoch in range(epochs):
    # Train on the training set using mlp.partial_fit
    mlp.partial_fit(X_train, y_train) ## YOUR CODE HERE

    # Compute training loss (MSE) for this epoch
    # using mlp.predict and the vector of true y training values
    train_loss = np.mean((mlp.predict(X_train) - y_train) ** 2) ## YOUR CODE
    train_losses.append(train_loss)

    # Compute validation loss (MSE) using mlp.predict and the vector of true
    val_loss = np.mean((mlp.predict(X_val) - y_val) ** 2) ## YOUR CODE HERE
    val_losses.append(val_loss)

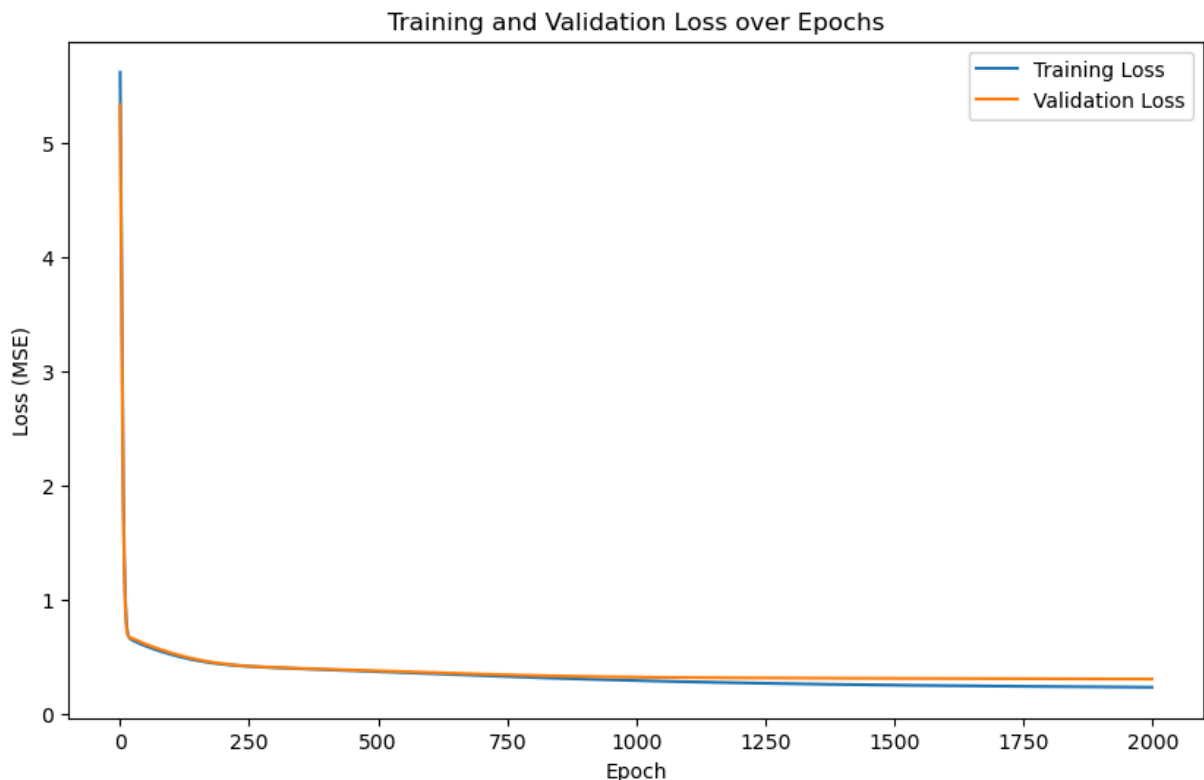
    # Print losses every 100 epochs
    if epoch % 100 == 0:
        print(f"Epoch {epoch}, Train Loss: {train_loss}, Validation Loss: {val_loss}")

# Plot the losses!
plt.figure(figsize=(10, 6))
plt.plot(train_losses, label="Training Loss")
plt.plot(val_losses, label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.title("Training and Validation Loss over Epochs")
plt.legend()
plt.show()

test_loss = np.mean((mlp.predict(X_test) - y_test) ** 2)
print(f"Test Loss after 2000 epochs: {test_loss}")

```

Epoch 0, Train Loss: 5.6222053528194875, Validation Loss: 5.338339811983645
Epoch 100, Train Loss: 0.516493005637783, Validation Loss: 0.530524215222263
8
Epoch 200, Train Loss: 0.4312997592665966, Validation Loss: 0.43640194502653
09
Epoch 300, Train Loss: 0.4017376530965469, Validation Loss: 0.40403324774970
72
Epoch 400, Train Loss: 0.38494530142605815, Validation Loss: 0.3884939065649
04
Epoch 500, Train Loss: 0.36788494692528145, Validation Loss: 0.3742855058320
1975
Epoch 600, Train Loss: 0.3497687482595643, Validation Loss: 0.35966868384617
906
Epoch 700, Train Loss: 0.33200940315919686, Validation Loss: 0.3457323731905
894
Epoch 800, Train Loss: 0.31578714412197306, Validation Loss: 0.3339562597683
5724
Epoch 900, Train Loss: 0.3014870420762151, Validation Loss: 0.32509333148089
58
Epoch 1000, Train Loss: 0.2890184047390207, Validation Loss: 0.3189965561932
8093
Epoch 1100, Train Loss: 0.27821728114563704, Validation Loss: 0.315054136811
7366
Epoch 1200, Train Loss: 0.26896668059300066, Validation Loss: 0.312568711067
94154
Epoch 1300, Train Loss: 0.2611429208068328, Validation Loss: 0.3109324589238
3814
Epoch 1400, Train Loss: 0.25456928530508144, Validation Loss: 0.309693949897
9554
Epoch 1500, Train Loss: 0.24903009632192127, Validation Loss: 0.308570785134
3997
Epoch 1600, Train Loss: 0.2443106891066332, Validation Loss: 0.3074234209647
4207
Epoch 1700, Train Loss: 0.24022710945732031, Validation Loss: 0.306210399635
2768
Epoch 1800, Train Loss: 0.2366354082933535, Validation Loss: 0.3049440809551
2554
Epoch 1900, Train Loss: 0.23342709565742883, Validation Loss: 0.303656274402
8353



Test Loss after 2000 epochs: 0.31068822019143316

🍓 Answer: The training loss is slightly lower (0.303 vs. 0.306), and the model's error on the test data is also very close to both the training and validation losses. This suggests that the model generalizes well and is not overfitting, since its performance is consistent across training, validation, and test sets.

? 3.4 Increase or decrease the number of neurons in the second layer, and experiment with adding more layers to the model. *Adjust the learning rate to make sure you aren't learning too quickly or too slowly.* Can you get the model to overfit to the training data? Explain how you know from your plot that you're overfitting. If you continue to iterate, the neural network will continue to use gradient descent to minimize loss on the training data. Do the train and test datasets have the same underlying loss function?

In [121... `from sklearn.neural_network import MLPRegressor`

```
mlp = MLPRegressor(
    hidden_layer_sizes=(50, 30, 25),
    activation='logistic',
    learning_rate_init=0.005,
    solver='sgd',
    max_iter=1,
    warm_start=True,
    random_state=12
) ## YOUR CODE HERE
```

```
# Number of epochs
epochs = 2000
```

```
# create empty arrays to store training and testing losses for each epoch
train_losses = []
val_losses = []

# Training loop
for epoch in range(epochs):
    # Train on the training set using mlp.partial_fit
    mlp.partial_fit(X_train, y_train) ## YOUR CODE HERE

    # Compute training loss (MSE) for this epoch
    # using mlp.predict and the vector of true y training values
    train_loss = np.mean((mlp.predict(X_train) - y_train) ** 2) ## YOUR CODE
    train_losses.append(train_loss)

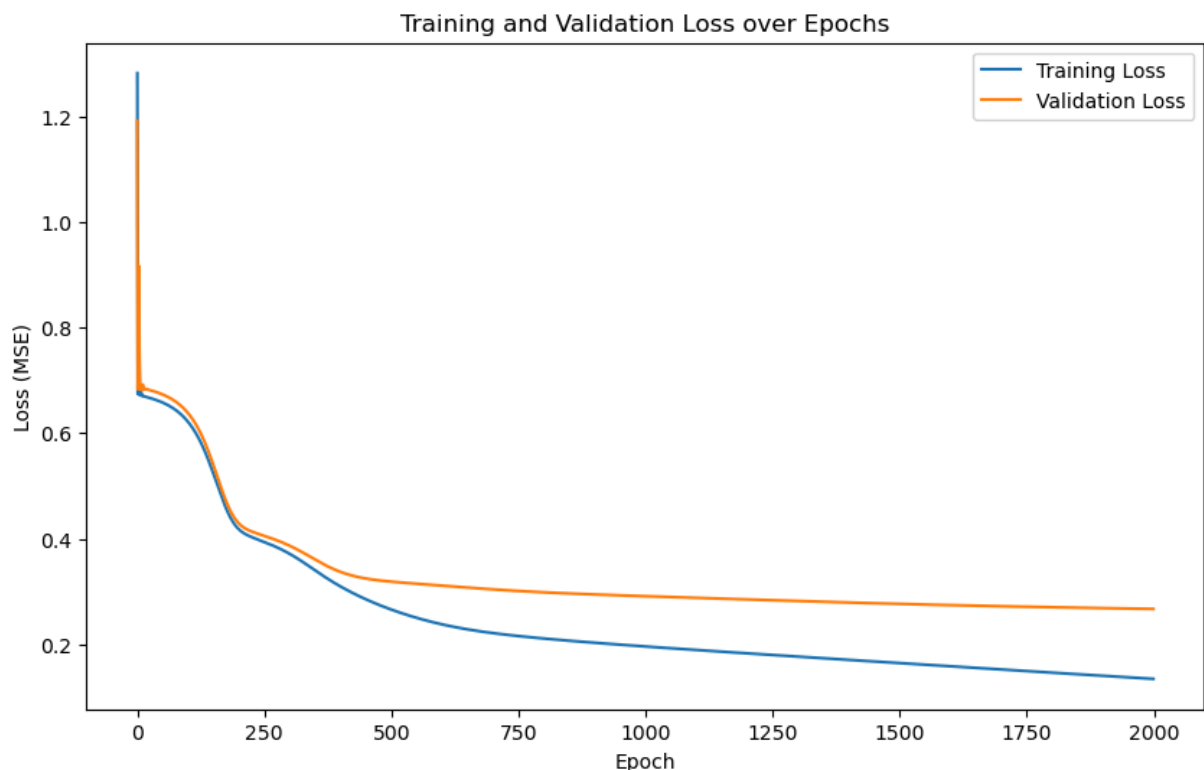
    # Compute validation loss (MSE) using mlp.predict and the vector of true
    val_loss = np.mean((mlp.predict(X_val) - y_val) ** 2) ## YOUR CODE HERE
    val_losses.append(val_loss)

    # Print losses every 100 epochs
    if epoch % 100 == 0:
        print(f"Epoch {epoch}, Train Loss: {train_loss}, Validation Loss: {val_loss}")

# Plot the losses!
plt.figure(figsize=(10, 6))
plt.plot(train_losses, label="Training Loss")
plt.plot(val_losses, label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.title("Training and Validation Loss over Epochs")
plt.legend()
plt.show()

test_loss = np.mean((mlp.predict(X_test) - y_test) ** 2)
print(f"Test Loss after 2000 epochs: {test_loss}")
```

Epoch 0, Train Loss: 1.281367067341774, Validation Loss: 1.1911225549359679
Epoch 100, Train Loss: 0.6210272365316426, Validation Loss: 0.6375646665948471
Epoch 200, Train Loss: 0.4176498619751337, Validation Loss: 0.4287093162381063
Epoch 300, Train Loss: 0.37129264043193155, Validation Loss: 0.386688735148458
Epoch 400, Train Loss: 0.3095333365961097, Validation Loss: 0.33718822485071687
Epoch 500, Train Loss: 0.265804219882061, Validation Loss: 0.31875904295818785
Epoch 600, Train Loss: 0.23782637201330667, Validation Loss: 0.3111444635416446
Epoch 700, Train Loss: 0.2213758181522854, Validation Loss: 0.3041308670733467
Epoch 800, Train Loss: 0.21111273136744194, Validation Loss: 0.2985879072766174
Epoch 900, Train Loss: 0.20312583947634022, Validation Loss: 0.29453728037194477
Epoch 1000, Train Loss: 0.19604778561758662, Validation Loss: 0.2912213910641637
Epoch 1100, Train Loss: 0.18943516201911695, Validation Loss: 0.2881393070624834
Epoch 1200, Train Loss: 0.18308148401820917, Validation Loss: 0.28510316378322614
Epoch 1300, Train Loss: 0.17686825752172994, Validation Loss: 0.28211643039148115
Epoch 1400, Train Loss: 0.17072846047505594, Validation Loss: 0.27926998829569055
Epoch 1500, Train Loss: 0.16462916911074782, Validation Loss: 0.2766616833473632
Epoch 1600, Train Loss: 0.15856186063405608, Validation Loss: 0.27433134623043715
Epoch 1700, Train Loss: 0.15252930238961118, Validation Loss: 0.27224599808374433
Epoch 1800, Train Loss: 0.1465330241484041, Validation Loss: 0.2703593266453426
Epoch 1900, Train Loss: 0.14058168686847008, Validation Loss: 0.268671611074546



Test Loss after 2000 epochs: 0.30580086445495114

✅ Answer: By increasing the model complexity, the network begins to overfit. This is observed in the plot where the training loss continues to decrease steadily, while the validation loss decreases more slowly and begins to plateau. The growing gap between training and validation loss indicates that the model is fitting the training data more closely than the validation data. Additionally, the test error (≈ 0.305) is higher than the training loss, further suggesting reduced generalization.

★ 3.5 Bonus - Go back to your original parameters from the beginning of the section. We've been using stochastic gradient descent, but with this function and this dataset that defaults to a batch size of 200. Try making the batch size smaller (try 32) or larger. What effect does this have on your model training? Try adjusting your learning rate to be an order of magnitude smaller to reduce overfitting when your batch size is small. Why might we want to use smaller learning rates with smaller batch sizes?

In [122... `from sklearn.neural_network import MLPRegressor`

```
mlp = MLPRegressor(
    hidden_layer_sizes=(50, 5),
    activation='relu',
    learning_rate_init=0.0001,
    solver='sgd',
    max_iter=1,
    batch_size = 32,
    warm_start=True,
    random_state=12
) ## YOUR CODE HERE
```

```
# Number of epochs
epochs = 2000

# create empty arrays to store training and testing losses for each epoch
train_losses = []
val_losses = []

# Training loop
for epoch in range(epochs):
    # Train on the training set using mlp.partial_fit
    mlp.partial_fit(X_train, y_train) ## YOUR CODE HERE

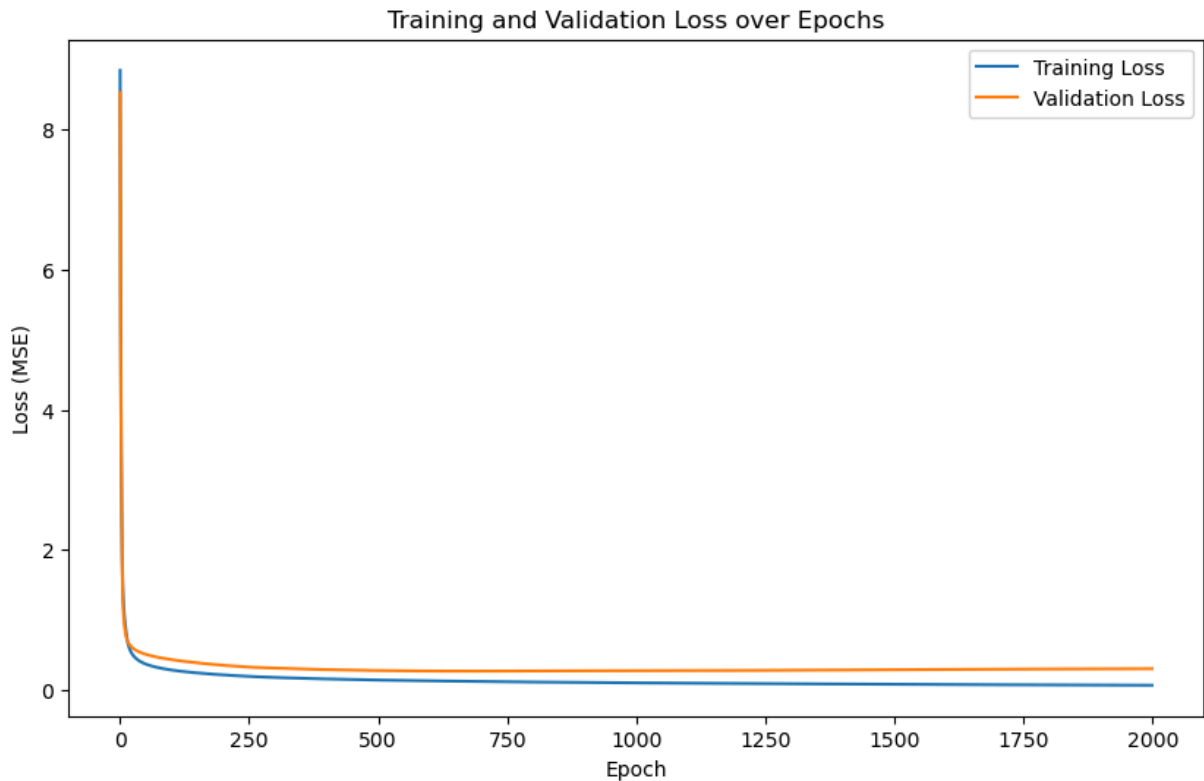
    # Compute training loss (MSE) for this epoch
    # using mlp.predict and the vector of true y training values
    train_loss = np.mean((mlp.predict(X_train) - y_train) ** 2) ## YOUR CODE HERE
    train_losses.append(train_loss)

    # Compute validation loss (MSE) using mlp.predict and the vector of true y validation values
    val_loss = np.mean((mlp.predict(X_val) - y_val) ** 2) ## YOUR CODE HERE
    val_losses.append(val_loss)

    # Print losses every 100 epochs
    if epoch % 100 == 0:
        print(f"Epoch {epoch}, Train Loss: {train_loss}, Validation Loss: {val_loss}")

# Plot the losses!
plt.figure(figsize=(10, 6))
plt.plot(train_losses, label="Training Loss")
plt.plot(val_losses, label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.title("Training and Validation Loss over Epochs")
plt.legend()
plt.show()
```


Epoch 0, Train Loss: 8.844111706862918, Validation Loss: 8.525642491452764
Epoch 100, Train Loss: 0.2865246379136591, Validation Loss: 0.4345431939373387
Epoch 200, Train Loss: 0.21722807681048528, Validation Loss: 0.3545506165387603
Epoch 300, Train Loss: 0.1809587192764688, Validation Loss: 0.3140973141516067
Epoch 400, Train Loss: 0.15966757802964446, Validation Loss: 0.2928840405284592
Epoch 500, Train Loss: 0.14434151950196017, Validation Loss: 0.2797100151889148
Epoch 600, Train Loss: 0.13258519712263922, Validation Loss: 0.2725898680298261
Epoch 700, Train Loss: 0.12336588567183168, Validation Loss: 0.27148418549224623
Epoch 800, Train Loss: 0.11580297766551659, Validation Loss: 0.27304764994561065
Epoch 900, Train Loss: 0.10948542173422864, Validation Loss: 0.27556399755671607
Epoch 1000, Train Loss: 0.1040935022083781, Validation Loss: 0.27763819529089884
Epoch 1100, Train Loss: 0.09933660268924714, Validation Loss: 0.2792212440000332
Epoch 1200, Train Loss: 0.09491785532440694, Validation Loss: 0.28089364225868746
Epoch 1300, Train Loss: 0.09088032443649251, Validation Loss: 0.2836392266735185
Epoch 1400, Train Loss: 0.0870644921586405, Validation Loss: 0.2867607499889781
Epoch 1500, Train Loss: 0.08357543523665614, Validation Loss: 0.290307301448714
Epoch 1600, Train Loss: 0.08024528581399978, Validation Loss: 0.2951237892121148
Epoch 1700, Train Loss: 0.07732808795181689, Validation Loss: 0.29859801485000553
Epoch 1800, Train Loss: 0.07465480514767497, Validation Loss: 0.30118296793353766
Epoch 1900, Train Loss: 0.07220885901232825, Validation Loss: 0.3035523221237723



🍓 Answer: The original model used a default batch size of 200. By explicitly reducing it to 32, the training becomes noisier due to less accurate gradient estimates. Smaller batch sizes introduce noise in the gradient, so we use smaller learning rates to prevent unstable or overly large updates.

Section 4 - Bonus

★ A long time ago in this assignment, we split off both a validation and test set of data! Use the held-out test data to compare at least 2 models (they don't have to both be neural networks, but they could be). Use plot comparing training and validation error to show that you selected reasonable (not overfit) models before comparing on the test set.

In []:

🍓 Briefly explain your results:

In []: